

onedoor

User Manual — the policy engine and the Policy Studio

"The model proposes; the policy layer disposes."

Covers onedoor 0.7.0: the decision engine (unchanged from the published 0.6.2) and the Policy Studio v2, including the upload track and the draft API. September 2026 · 0.7.0 edition.

1 · What onedoor is

onedoor is a tiered guardrail engine for agentic systems. Every action your software wants to take — scheduled, rule-fired, LLM-proposed, or human-clicked — is submitted as a structured **ActionRequest** and evaluated by one decision pipeline before anything touches the world. Nothing executes around it; that is the name.

The three-minute mental model

One door	Callers never execute directly; they submit an ActionRequest (action type + parameters + rationale) to the engine.
Ordered checks	Kill switch → policy lookup (default-deny for unknown action types) → tier-1 integrity (no undo → no autonomy) → bounds → dry-run → caps (reserved race-free) → intent recorded.
Four outcomes	<i>Permitted</i> (an obligation: you enforce, then report) · <i>denied</i> (with a typed reason) · <i>proposed</i> (waiting for a human approval) · <i>dry-run</i> (logged, nothing happens).
Everything audited	An append-only log: decisions, results, denials, approvals, kill-switch blocks — every row carries a reason code.

The most load-bearing fact: an action type you have not written a policy for cannot auto-execute. Absence is denial — unknown actions land at Tier 3, propose-and-confirm. An empty policy table means *nothing is permitted*, and the Studio says so on its face.

2 · Core concepts

Autonomy tiers

Tier	Name	Meaning
0	Observe	Audited reads. Logged, allowed, never reach connectors; exempt from the kill switch.
1	Auto	Auto-executes — only if reversible (a compensating command exists), in bounds, with an undo window.
2	Auto-capped	Auto-executes under caps: a rate per day plus cumulative money budgets (per day / per month).
3	Confirm	Propose-and-confirm: a human approves, with a TTL. Where every unknown action type lands.

Tiers attach to **action types**, not to agents. There is deliberately no deny tier: denial is the default, and a rule's job is to grant, never to forbid.

Drafts vs. policy versions

A **draft** is a policy set that has not been ratified: the only editable, non-binding object in the system — a bill, not a law. The engine never reads drafts. Ratifying a draft (through the Studio's ceremony) turns it into a **numbered, immutable policy version** — the thing the engine enforces and receipts attest. Versions are never edited, only succeeded: to change policy, draft again and ratify version $n+1$; version n stands in the record forever. Everything mutable has no authority; everything with authority is immutable.

A draft has two states, **draft** and **submitted** — and deliberately no *ratified* state: ratification is a receipt in the engine's store, never a property of the draft. Submitting means *a human has been asked*; it moves no version pointer and writes no receipt. Every draft is also **pinned** to the version that was in force when its numbers were computed. If a newer version is ratified underneath it, the draft is marked stale and cannot

be submitted — every number computed for it is about a dead base — until you re-pin it, which recomputes everything against the version now in force.

Receipts

Every ratification produces a receipt: a small record binding the ratified version to a content digest. A receipt can be verified **offline**, with a command that reads two files and opens no database (§10). The Studio shows you a verification and tells you how to repeat it without trusting the Studio.

3 · Getting started

Install (Windows paths shown; on macOS/Linux use `.venv/bin/...`)

```
python -m venv .venv
.venv\Scripts\pip install -e ".[studio]"      # from a repo checkout
# or, from PyPI:
.venv\Scripts\pip install "onedoor[studio]"
```

Check, then start the Studio

```
python -m onedoor.studio --help
python -m onedoor.studio --db onedoor.db --studio-db studio.db --port 8787
```

Then open **`http://127.0.0.1:8787`**.

Two things to know before anything else. (1) **--host accepts loopback and nothing else** — there is no flag to expose the Studio to a network, by design. (2) **Name --db explicitly**. The decision service defaults to *onedoor-service.db* and the Studio to *onedoor.db*; accept both defaults and they point at different stores — the Studio comes up, works, and shows you an empty world. A fresh store says so on its face, naming both defaults.

4 · The Policy Studio — the eight screens

Screen	What it is for
Drafts	Where policy sets are born — typed, uploaded from a file, or created over the API. The landing page; even empty it offers a form and a curl one-liner.
Policies	What is actually in force. Empty reads “nothing is permitted” — absence is denial.
History	The execution ledger: every decision, chain-numbered so a citation survives the view, with the policy version that made it and its reason code.
Live state	Budgets (consumed = counter – reserved) and the kill switch — shown here, thrown only via the admin API (capability is not authority), with what the switch does <i>not</i> stop stated on the page.
Editor	Guided pane and raw pane rendered from one server-side parser — they cannot disagree. Saving nonsense refuses, says the draft is unchanged, and writes nothing.
Ceremony	Ratification. A page before it is an action: what will be in force, what changes, what ratifying does not undo — in the ceremony's own words.
Replay	Open any past decision and re-evaluate it under a different policy version. Calls the real engine and answers in the conditional: what <i>would have</i> happened.
Verify	The deposition page — method first, answer last: the receipt, the two files with Download links serving the exact stored bytes, and the offline command that checks them without trusting the Studio.

5 · Writing policy

There are three ways to author policy, and all three land in the same place — a draft, which changes nothing until a human ratifies it: the **editor** (this section), an **uploaded file** (§6), and the **API** (§7).

Policies are data, not code. Under the hood a policy set is YAML; the Studio's editor reads and writes the same object its raw pane shows. A minimal rule binds an action type to a tier; a production rule usually carries reversibility, bounds, and caps:

```
policies:
- action_type: payments.transfer      # exact match on ActionRequest.action_type
  tier: 3                             # 0 observe | 1 auto | 2 auto-capped | 3 confirm
  dry_run: true                       # default true: new action types rehearse first
  compensating_command: payments.reverse # REQUIRED for tiers 1 and 2
  undo_window_seconds: 900
  bounds:
    numeric: { amount_eur: { min: 0.01, max: 500 } }
    enum:    { currency: [EUR] }
    required: [payee, amount_eur]
    strict_params: true               # reject undeclared params (injection defense)
  caps:
    daily_rate: 20                   # max executions per local day
    eur_day:    "50.00"              # tier-2 cumulative budgets (decimal strings)
    eur_month:  "500.00"
```

Field notes that save you an afternoon

Field	What to know
dry_run / dry_run_until	A dry-run resolves <i>before</i> caps — rehearsals never spend budget. The two fields are ORed: dry_run: true rehearses indefinitely and a date will never switch it live; to rehearse until a date, set dry_run: false and give the date. Dry-run governs the automatic path only: a Tier-3 action in dry-run still creates a real approval, and approving executes for real.
compensating_comm and	The reversibility rule. Must name another registered action type; the undo runs through the same pipeline, linked to its parent. Every auto-executing tier needs one: a rule without it fails at load, and an auto action that loses its reversal demotes to Tier 3 at runtime.
bounds	Validated for every tier before proposals are created, so approvers only ever see sane requests. strict_params: true rejects any parameter not named.
caps + cost_param	daily_rate counts executions, reserved at decide time (race-free). Euro caps apply to Tier 2 and need to know which parameter carries the money: any action with a euro cap needs a cost_param (or a caller setting cost_eur). An amount the engine cannot resolve is a denial with reason cost_unknown — never an assumed zero.

Effects — governance that survives renaming

Policy binds to action *names*, but the same real-world effect is reachable through many tools (a payments tool, or a generic HTTP tool aimed at the bank). Effects give the engine a second, name-independent binding: label actions (or match parameters deterministically — a regex on a string, or canonicalized URL rules for hosts, subdomains, CIDRs) with an effect such as money.egress, then set a tier floor and one shared budget on the effect itself. Every carrier draws from the same cap; escalations carry reason effect_floor. A URL the canonicalizer cannot interpret is denied as malformed — a parse differential is a denial, never a bypass — and declared redirector hosts are never auto-executed: a human approves, or policy denies.

6 · Uploading a policy file — refusals as answers

On **Drafts** → **From a file**, upload a YAML policy file. The bytes enter at the top of the engine's own loader path and pass through four named stages: *reading the file* → *checking each rule against the schema* → *applying the per-rule rules* → *building the effect policies*. **A file the loader would refuse still creates a draft** — nothing is repaired and nothing is guessed; the draft holds whatever parsed, and its page shows the refusals with the stage that stopped and, where the file allows one, a line and column. The uploaded bytes are frozen verbatim before anything parses them, so what you actually sent survives independently of what this build could make of it.

Reading the refusal page

Panel	What it tells you
The loader would refuse this	What fails at startup, in the loader's own words, with the stage named. Below it: which later stages did <i>not run</i> — silence from a check that never ran is not a pass, and the page says so.
Once in force, these rules will...	Forecasts, not refusals: how a rule that loads will behave at decision time, each line naming the reason code the engine will record. When something above is refused, this panel's own preamble says so — it never claims the loader accepts what the list above refuses.

A draft the loader would refuse cannot be ratified. Its Changes panel says it cannot be previewed, the ceremony page shows no Ratify button, and a direct POST to the ratify route is refused with reason *candidate_invalid_at_load* — the preview and the real ratification share one application path by design, so they refuse the identical candidate for the identical reason.

7 · The API — and what it deliberately cannot do

Everything under **/api/v1** speaks JSON and answers on loopback, same as the screens. Create and edit drafts (PUT one rule at a time — the other rules are carried over, never rebuilt), read both validation lists as data (refusals and forecasts are separate keys, never merged), and submit a draft for ratification. The schema is at `/api/v1/openapi.json`.

```
curl -X POST localhost:8787/api/v1/drafts \
  -H "Content-Type: application/json" \
  -d '{"title":"from the api",
      "rules":[{"action_type":"reports.read","tier":3}]}'
curl localhost:8787/api/v1/drafts/<id>/validation
curl -X POST localhost:8787/api/v1/drafts/<id>/submit
```

The v1 API adds no approval route. Submit sets a flag meaning a human has been asked and returns the ceremony page's address; no version pointer moves and no receipt is written. One legacy route (POST `/draft/{id}/ratify`), predating actor identity, still serves; it is marked deprecated in the schema itself, records its approver as declared — never authenticated — and is retired with the actor-identity work.

The model-assisted proposer (a draft from a plain-language description) is present but opt-in: it runs only against a model endpoint you configure, produces **drafts only**, and never ratifies. Its quality is measured by a deterministic benchmark that publishes both catches and gaps; improvement work is queued for 0.7.1.

8 · The working loop

1 · Draft	Create a draft on the Drafts screen; add rules in the editor.
2 · Ratify	Review and ratify — the ceremony shows what will be in force before you confirm. You now have a numbered version and a receipt.
3 · Wire	Point your agents at the engine (§9). Every action flows through decide-and-reserve and returns a terminal result with a reason code.
4 · Watch	History fills with chain-numbered decisions; Live state shows budgets and the switch.
5 · Replay, then change	Before ratifying a stricter version, open past decisions and re-evaluate under your candidate's predecessors: would this have been refused? Test policy against reality, then draft the amendment and ratify n+1.
6 · Verify	Download any receipt's two files and check them offline (§10). That is what you hand an auditor.

First decision on a fresh install — the smallest honest way to put a row in History:

```
python -m onedoor.studio.walkthrough --db onedoor.db
```

9 · Wiring your agents — pick one door

You have	Use
A Python codebase (agent, framework, backend)	Embed the library — call <code>decide_and_reserve</code> in-process (docs: integration-library).
Anything that speaks HTTP	Run the decision service and <code>POST /v1/decide</code> (docs: integration-service).
An MCP host + tool servers	The MCP proxy — tools gated transparently (docs: integration-mcp).
A LiteLLM proxy	The custom guardrail adapter (docs: integration-litellm).
A LangGraph / LangChain agent	Governed tool wrapper + interrupts (docs: integration-langgraph).

Whichever door: *permitted* is an obligation (you enforce, then report the result), *reserve-and-commit* means a crashed action never leaks budget, and two concurrent requests can never share the last slot under a cap.

10 · Verifying a receipt — without trusting the Studio

On the Verify screen, open a receipt and click **Download** on both panels — the links serve the exact stored bytes, so nothing is lost to copy-and-paste (one wrong byte would turn a sound receipt into a false *failed*). Then, on any machine:

```
python -m onedoor.studio.verify receipt.json snapshot.json
```

Output	Exit	Meaning
verified	0	The receipt matches its own digest and the snapshot hashes to the version the receipt ratified.
failed	1	The check ran and the answer is no: a readable snapshot whose bytes hash elsewhere, or a receipt that does not match its own digest. The output names the hash it actually got.

Output	Exit	Meaning
unreadable	2	A file could not be read or parsed. Not a failed check — a check that never ran.

The command reads two files and opens no database. Copy them to another machine and it gives the same answer — which is the entire point. One asymmetry worth knowing: the **receipt is parsed** (a truncated receipt is *unreadable* — the check never ran), but the **snapshot is only hashed**, byte for byte, against the version the receipt ratified — so any corruption of the snapshot, however small, is *failed*: a readable file whose bytes hash elsewhere is exactly what tampering looks like.

11 · Reason codes you will see

Code	Meaning
passed	Checks passed; the action was permitted.
default_deny	No policy for this action type — unknown actions cannot auto-execute.
tier_confirm	Tier 3: proposed, awaiting human approval.
no_compensating_command	An auto tier lost its reversal — demoted to confirm.
bounds	A parameter was out of bounds (or undeclared, under strict_params).
dry_run	Rehearsal: logged, nothing executed, no budget spent.
cap_rate / cap_value	A rate or money cap would be exceeded (window and unit in the budget object).
cost_unknown	A euro cap applies but the amount could not be resolved — denied, never assumed zero.
kill_switch	The switch is thrown; automatic execution blocked (Tier-0 reads exempt).
observe	Tier 0: recorded, allowed, no connector reached.
effect_floor	An effect label escalated the tier (e.g. money.egress carries a floor).
malformed	A target could not be canonicalized — a parse differential is a denial, never a bypass.
expired	An approval outlived its TTL.

Studio refusals (surface, not engine)

Reason	Meaning
base_moved	The draft is pinned to a version no longer in force; re-pin before submitting.
candidate_invalid_at_load	The candidate would be refused by the loader — ratification refuses it in the loader's own words.
malformed_request	The request body could not be read as JSON; the message carries the parser's own position.

12 · Design guarantees you can rely on

- An action type absent from the policy table **cannot** auto-execute (default-deny → Tier 3).
- A Tier-1 policy without a compensating_command **cannot** load, and cannot execute (runtime demotion to Tier 3).
- Bounds are validated before a proposal is created — approvers only ever see in-bounds requests.
- A dry-run never consumes caps.
- Two concurrent requests cannot share the last slot under a cap (check-and-reserve inside the deciding transaction).
- The audit log is append-only; results link to intents; an undo links to its parent.
- The Studio serves loopback only, and ratified versions are immutable — changes are new versions, never edits.
- A draft the loader would refuse **cannot** be ratified — the preview, the ceremony page and the ratify route refuse the identical candidate for the identical reason.

- Submitting is asking, not approving: no version pointer moves and no receipt is written until a human confirms on the ceremony page.

13 · When something looks wrong

Symptom	What it means
The Studio shows an empty world you know is not empty	It opened a different store than your service. Restart with --db naming the engine's database explicitly; the fresh-store warning names both defaults.
A rule refuses to load	Most often a tier-1/2 rule without a compensating_command — that is a boot failure by design, not a bug.
An action you allowed was denied cost_unknown	The euro cap could not find the amount: set cost_param on the rule (and list that parameter under bounds.required), or have the caller set cost_eur.
A dry-run action “did nothing”	That is the contract: logged, no execution, no budget. For Tier 3, dry-run still proposes — approving executes for real.
verify prints unreadable	A file problem, not a verification failure — usually a receipt that no longer parses. Re-download the two files and run again.
verify prints failed on files you copied by hand	The snapshot is hashed byte-for-byte; one byte lost in transit fails honestly. Use the Download links, never select-and-paste.
Submit answers base_moved	A newer version was ratified after this draft was pinned. Re-pin the draft; its numbers recompute against the version now in force.
You need the Studio on another machine	You cannot — loopback only, no override flag. Use an SSH tunnel if you must view it remotely.
Write down anything that surprises you, in your own words, including things that merely felt wrong. The findings that shaped this Studio most came from someone using it and saying what happened.	